HR Roster Sync

Idempotent, Out-of-Order, Self-Healing | Java backend submission

CANDIDATE	SUBMISSION	LINKS
Ayush Vishwakarma imt_2022028@iiitm.ac.in	Case Study 5 Java 21 / Spring Boot PostgreSQL	Source repository Five-minute demo video

Problem and outcome

MoveInSync must combine near-real-time HR events with nightly exports of up to 200,000 employees. Inputs can be duplicated, delayed, invalid or truncated. HR-owned fields must remain separate from transport-owned preferences, and employee deactivation must reliably cancel future bookings and release seats.

The implementation validates each record independently, applies only newer source state, streams CSV data into staging, blocks suspiciously incomplete snapshots, and commits every employee change with an outbox event in the same database transaction.

Architecture

HR source	Authenticated API	Decision layer	PostgreSQL	Outbox worker
Live events Nightly CSV	Role + tenant Boundary validation	Ordering Ownership Safety brake	Employee + staging Outbox + inbox	Booking cancel Seat release

Component responsibilities

COMPONENT	TECHNICAL RESPONSIBILITY
Controller / security	HTTP boundary, per-record errors, role checks, authenticated tenant identity.
Service / strategy	Ordering and ownership decisions; transaction boundaries; configurable version/timestamp strategy.
Repository	Tenant-scoped SQL, locking, staging, reconciliation diff, employee/outbox/inbox persistence.
Outbox worker	Durable retries and idempotent booking effects after an employee transaction commits.
Dashboard / metrics	Read-only operational view, protected Actuator counters, structured privacy-safe logs.

Core patterns

Repository, Strategy, Transactional Outbox, Idempotent Consumer/Inbox, Staging Table, Cache-aside and Dependency Injection. Encapsulation uses immutable domain/request records and private collaborators; polymorphism is used only where ordering behavior genuinely varies.

Correctness and recovery

The hard rules are enforced at the database transaction boundary and covered by automated tests.

Ordering, idempotency and ownership

RULE	IMPLEMENTED GUARANTEE
Ordering	VERSION_PREFERRED uses numeric version when present, otherwise source timestamp. VERSION_ONLY and TIMESTAMP_ONLY are startup-configurable. Equal or older state is ignored.
Idempotency key	Tenant + employee ID + ordering kind + source value + event type. State comparison is the final guard, so replay is a no-op even after a retry or restart.
Field ownership	HR owns identity, manager, office, shift and active status. Transport owns pickup point and assistance. HR update SQL never writes transport-owned columns.
Tenant isolation	Tenant comes from authenticated authorities, never request parameters. Every read/write query includes the tenant key.

Nightly reconciliation

1. Stream CSV rows into bounded staging batches; persist row-level validation errors.
2. Lock the tenant and compare staged hashes with stored HR hashes in PostgreSQL.
3. Recalculate missing active employees. If missing / active_before is at least the configurable threshold (default 30%), persist **BLOCKED** and apply no employee changes.
4. On operator approval, lock and recalculate against current data.
5. Apply newer rows, eligible absence deactivations and outbox entries; then advance the cutoff and complete the run in one transaction.

A PARTIAL run may apply valid rows but never treats absence as deactivation and never advances the snapshot cutoff. Staging grows with the file; application memory is bounded by 1,000-row buffers.

Failure behavior

FAILURE POINT	VISIBLE BEHAVIOR	RECOVERY
Invalid row	Per-record rejection; valid records continue.	Correct source data and resend.
Crash during staging	Employee state is unchanged.	Re-upload; incomplete staging is retained.
Crash before transaction commit	Employee, outbox and cutoff changes roll back together.	Resume the staged run.
Crash after commit, before publish	Pending outbox remains durable.	Worker resumes after restart.
Booking consumer failure	Inbox receipt and effects roll back; retry counters increase.	Backoff retry; operator can inspect/retry.

Security, monitoring and cache

BCrypt-backed demo accounts implement employee, HR, admin and operator roles. Production deployment would replace Basic authentication with OIDC/OAuth2 behind TLS. Stable JSON errors and structured logs omit phone, email, assistance and location data. Operator metrics expose outbox depth/lag, successes, failures, retries, rejections and blocked snapshots.

Only office-code validation is cached: tenant-keyed, maximum 10,000 entries, 60-second expiry. Employee/order/outbox state is never cached. Staleness is therefore limited to an office code being accepted for at most 60 seconds after administrative removal.

Reproduce and evaluate

One command starts the application and PostgreSQL. The same core flows are available through curl and the dashboard.

Quick verification

```
# Repository root / Windows PowerShell
$base="http://localhost:8080"; $hr="acme-hr:local-demo-password"
docker compose up --build -d
curl.exe "$base/actuator/health"

# APPLIED; run the same command again for IGNORED / STALE_OR_DUPLICATE.
curl.exe -u $hr -H "Content-Type: application/json" `
  --data-binary "@examples/events.json" "$base/api/events"

# One of two employees is missing: BLOCKED.
curl.exe -u $hr -F "file=@examples/snapshot.csv" `
  "$base/api/snapshots?watermark=10"

# No credentials: 401. Operator metric: numeric queue depth.
curl.exe -i "$base/api/employees"
curl.exe -u acme-operator:local-demo-password `
  $base/actuator/metrics/roster.outbox.pending
```

Expected signals

Health reports **UP**; the first event feed applies two employees; replay ignores both; the incomplete snapshot is **BLOCKED**; unauthenticated access returns **401**; the operator metric returns a numeric value. For deterministic first-run output, reset local data with `docker compose down -v` before starting.

Automated evidence

AREA	EVIDENCE
Build and tests	Maven verify passed: 26 tests, no failures or skips; Spotless and executable JAR packaging passed.
Hard rules	Duplicate/stale ordering, ownership, bad-row isolation, tenant/role isolation, safety brake and approval.
Advanced cases	Concurrency races, transaction rollback, outbox crash recovery, consumer retry and cache isolation/eviction.
Artifacts	README.md, DEMO_GUIDE.md, docs/design.md, docs/verification.md and docs/*-results.json.

Measured performance

WORKLOAD	RESULT	NOTES
200,000-row initial import	28.582 s / 6,997 rows/s	Includes upload, validation, staging, reconciliation and response.
200,000-row unchanged diff	16.965 s / 11,789 rows/s	Bounded JVM staging buffers; PostgreSQL holds staged rows.
100 API requests / 8 clients	2.623 s; p50 172.34 ms; p99 602.13 ms	Small local sample, not a capacity claim.

Machine: Windows 11, Intel Core i5-12500H, approximately 16 GiB RAM. JVM heap: 256 MiB; observed peak application working set: approximately 272 MiB. The import benchmark disabled the outbox worker and predates timestamp/dashboard extensions.

Limits and deliberate trade-offs

Tenant-wide locking favors correctness and deterministic ordering but serializes writes within a tenant. The 30% safety brake cannot detect every smaller corrupt export; a production source manifest/count/checksum would strengthen completeness. The in-process outbox worker proves durable local effects, while external service delivery would add a broker and publisher. Database staging and retained outbox history require production retention policies.

Submission links

Repository: https://github.com/Ayush-Vish/HR_Roster_Sync
Demo: Google Drive technical walkthrough